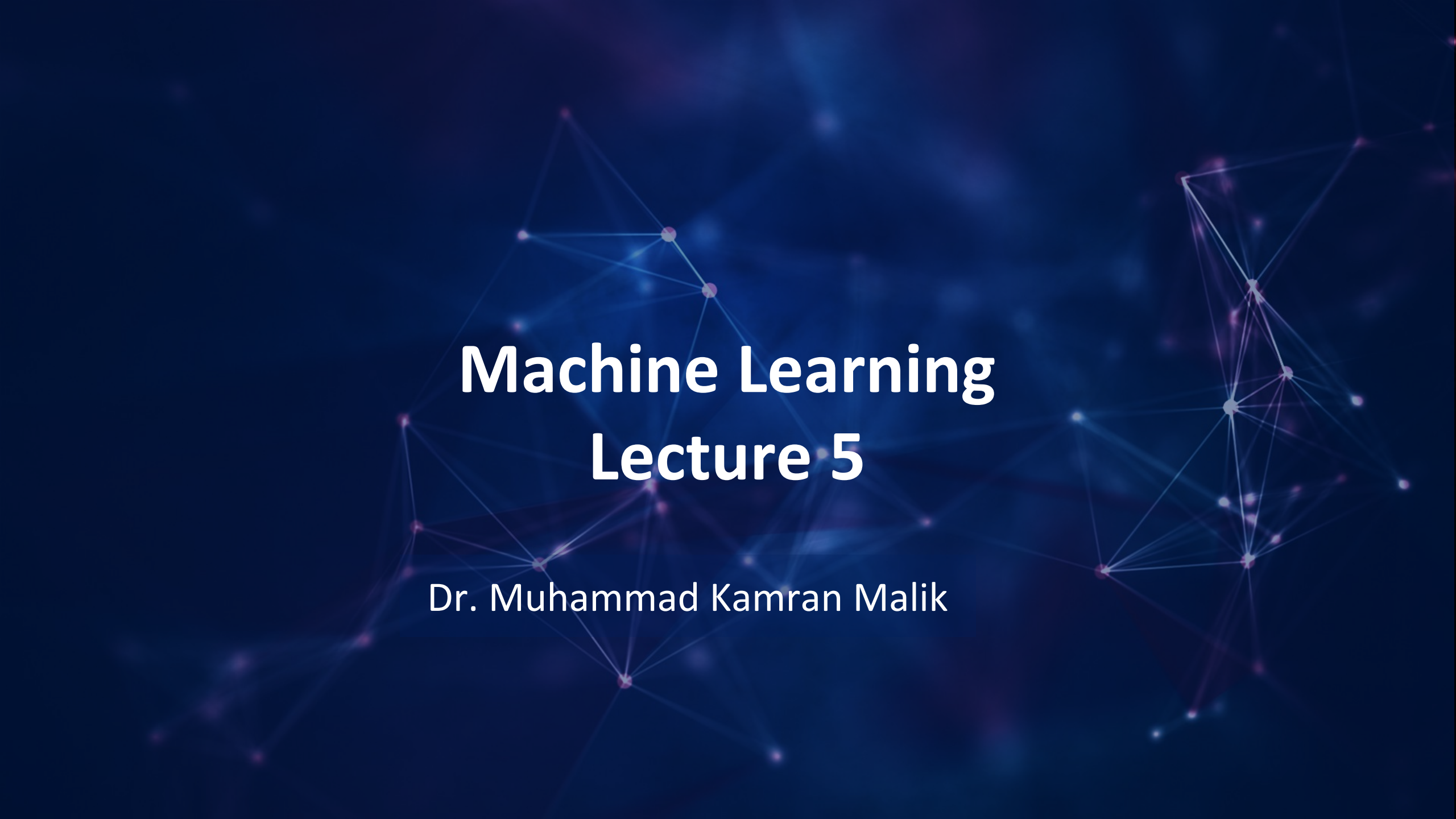


The background is a dark blue gradient with a complex network of glowing white and light blue lines connecting various points, resembling a neural network or a data visualization. The points are small circles, some of which are highlighted in a light blue or white color.

Machine Learning Lecture 5

Dr. Muhammad Kamran Malik



Training Linear Regression using Gradient Descent

Gradient Descent

Have some function $J(w, b)$ for linear regression or any function

Want $\min_b J(w, b)$ $\min_{w_1, \dots, w_n, b} J(w_1, w_2, \dots, w_n, b)$

Outline:

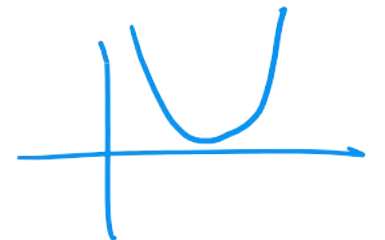
Start with some w, b (set $w=0, b=0$)

Keep changing w, b to reduce $J(w, b)$

Until we settle at or near a minimum

may have >1 minimum

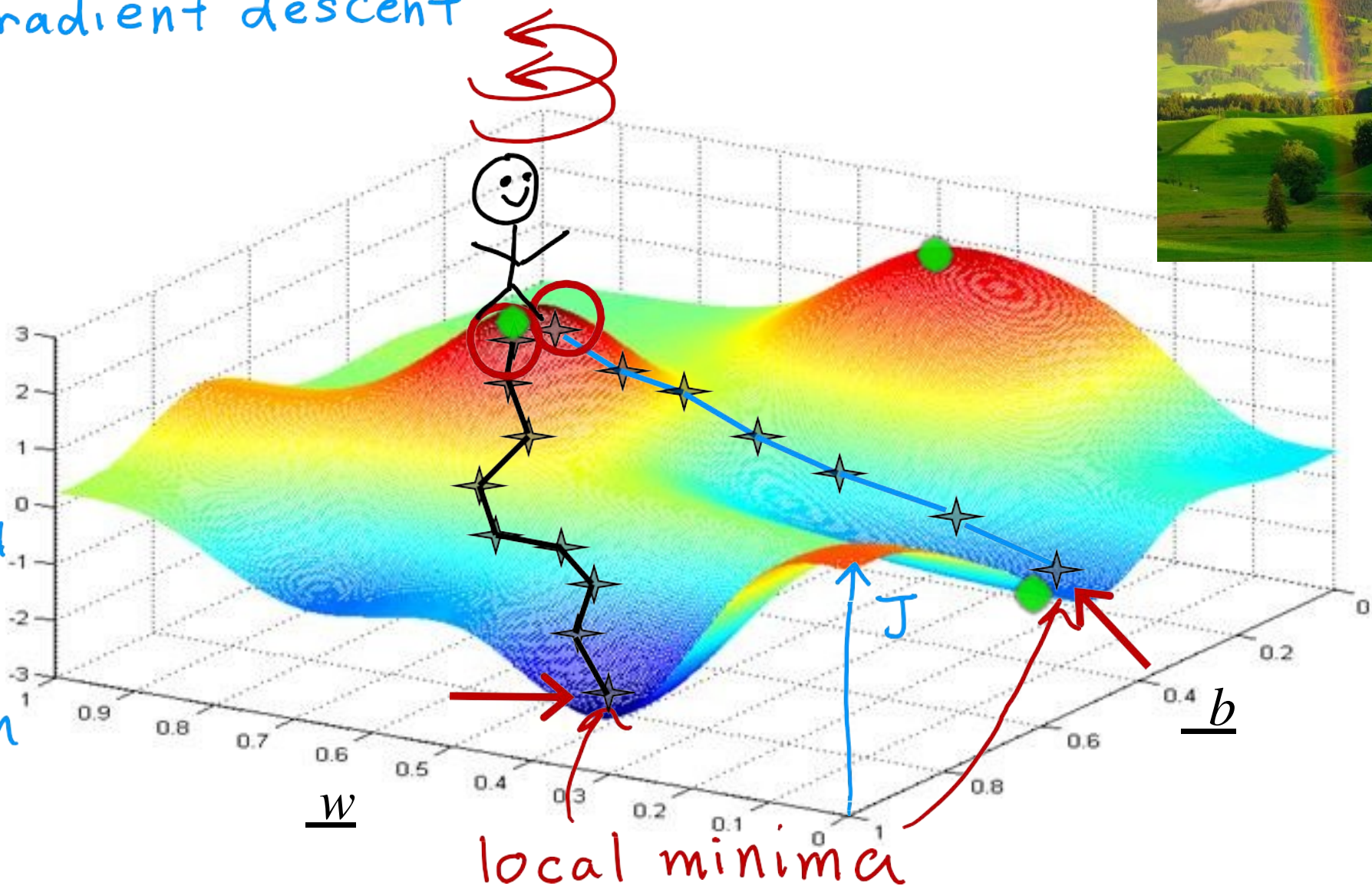
J not always



Gradient Descent

gradient descent

$J(w, b)$
not squared
error cost
not linear
regression





Implementing Gradient Descent

Gradient descent algorithm

Repeat until convergence

$$\left\{ \begin{array}{l} \underline{w} = w - \alpha \frac{d}{dw} J(w, b) \\ \underline{b} = b - \alpha \frac{d}{db} J(w, b) \end{array} \right.$$

Learning
rate
Derivative

Simultaneously
update w and b

Assignment

$$a = c$$

$$a = a + 1$$

Code

Truth assertion

$$a = c$$

$$a = a + 1$$

Math

$$a == c$$

Correct: Simultaneous update

$$\begin{array}{l} tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b) \\ tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b) \\ w = tmp_w \\ \underline{b} = tmp_b \end{array}$$

Incorrect

$$\begin{array}{l} tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b) \\ \underline{w} = tmp_w \\ tmp_b = b - \alpha \frac{\partial}{\partial b} J(\underline{w}, b) \\ \underline{b} = tmp_b \end{array}$$



Gradient Descent Intuition

Gradient descent algorithm

● repeat until convergence {

learning rate α $\rightarrow$ derivative

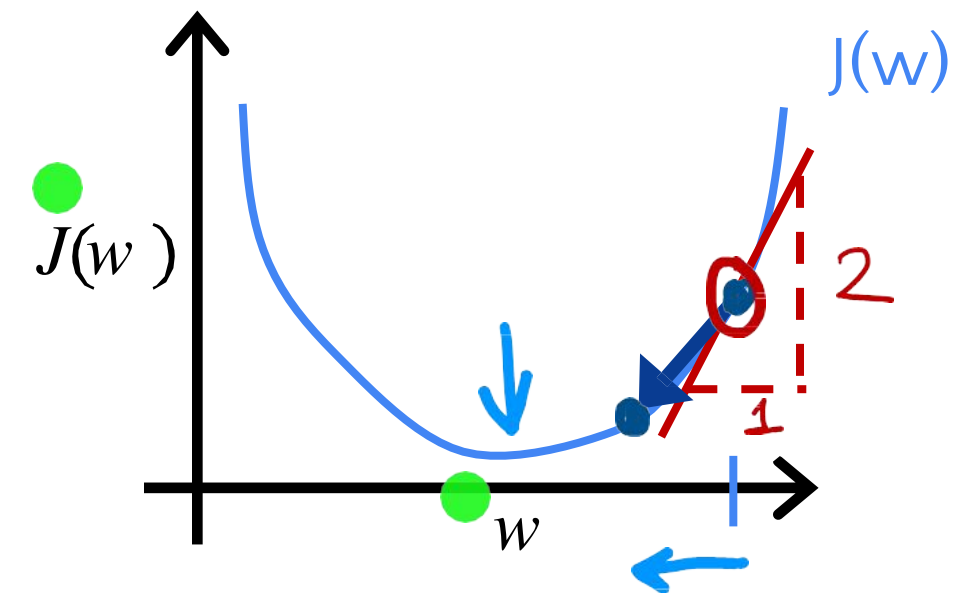
$$\begin{cases} \underline{w} = w - \alpha \frac{\partial}{\partial w} J(w, b) \\ \underline{b} = b - \alpha \frac{\partial}{\partial b} J(w, b) \end{cases}$$

● $J()$

$$\underline{w} = w - \alpha \frac{\partial}{\partial w} J(w)$$

min $J()$

~~w~~
w

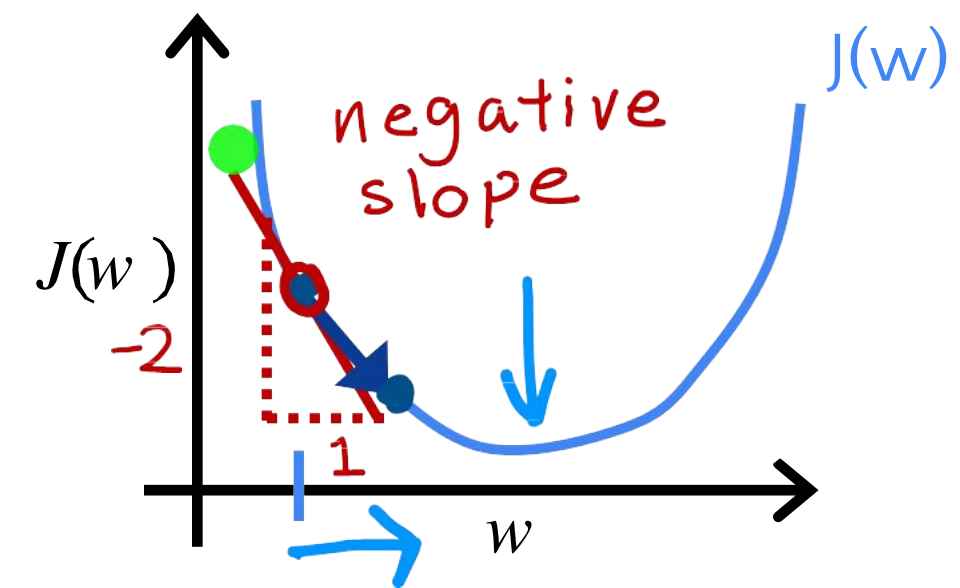


$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot \text{(positive number)}$$

$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

$$w = w - \alpha \cdot \text{(negative number)}$$





Learning Rate

$$w = w - \alpha \frac{d}{dw} J(w)$$



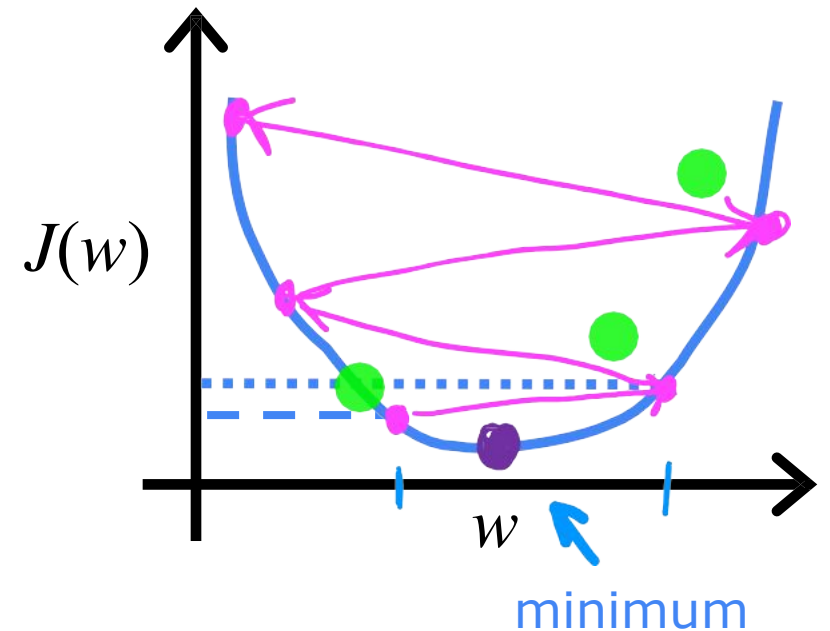
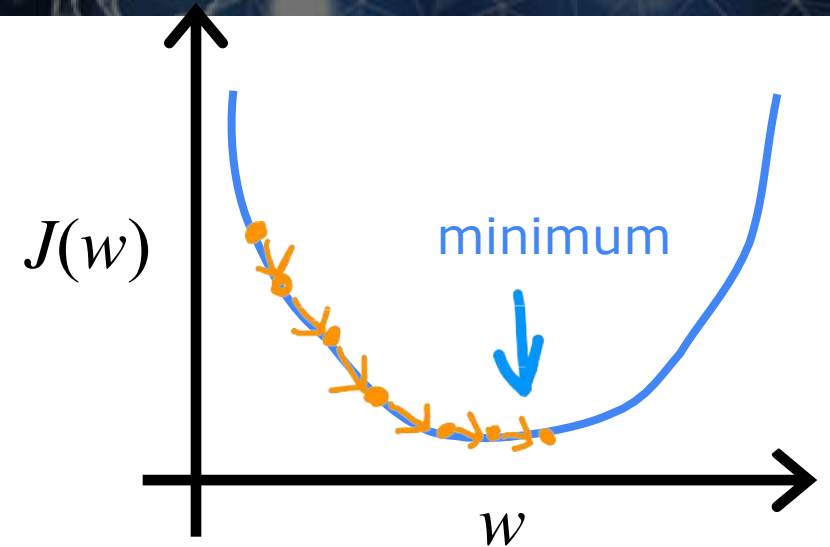
If α is too small...

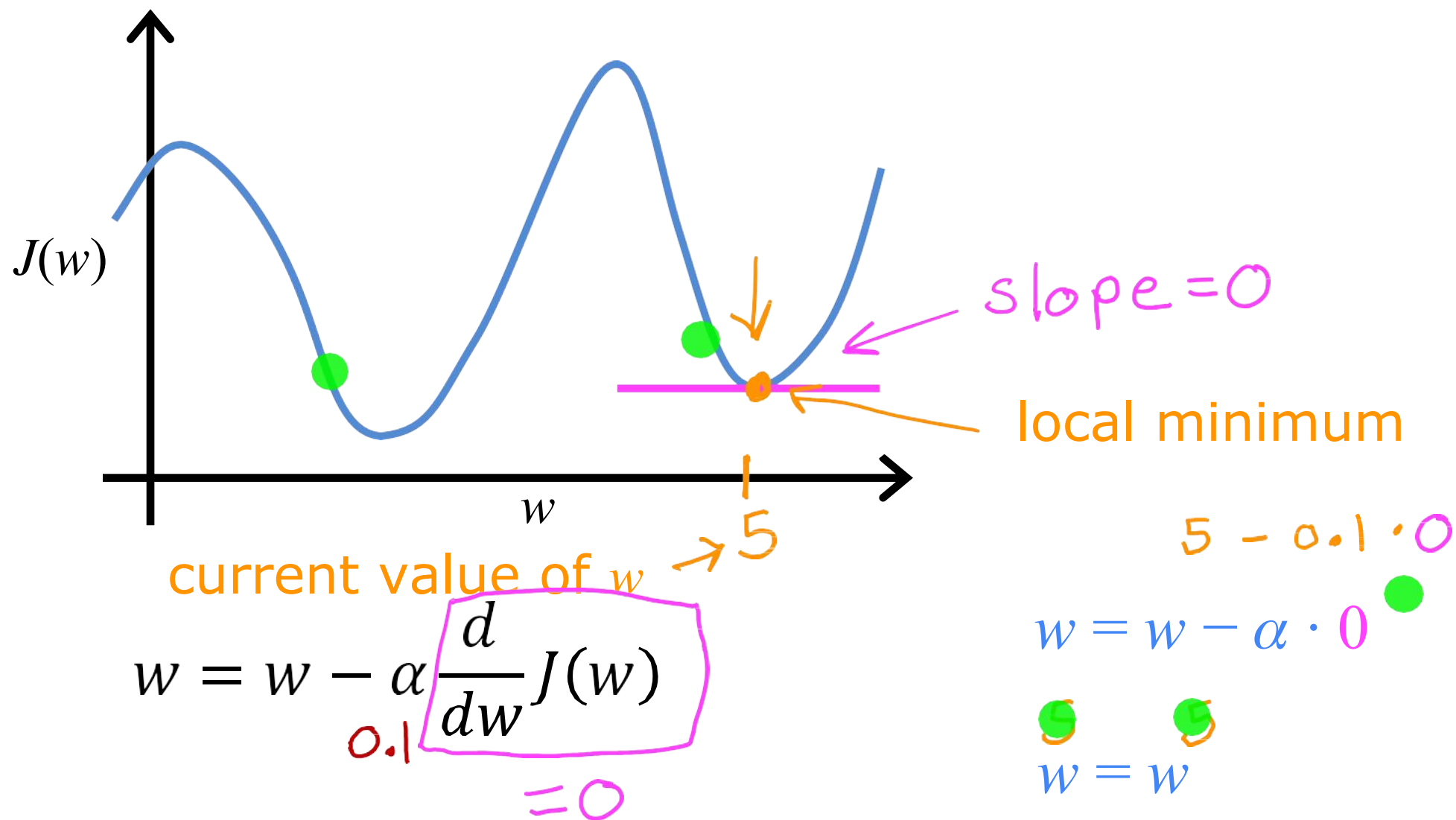
Gradient descent may be slow.

If α is too large...

Gradient descent may:

- Overshoot, never reach minimum
- Fail to converge, diverge





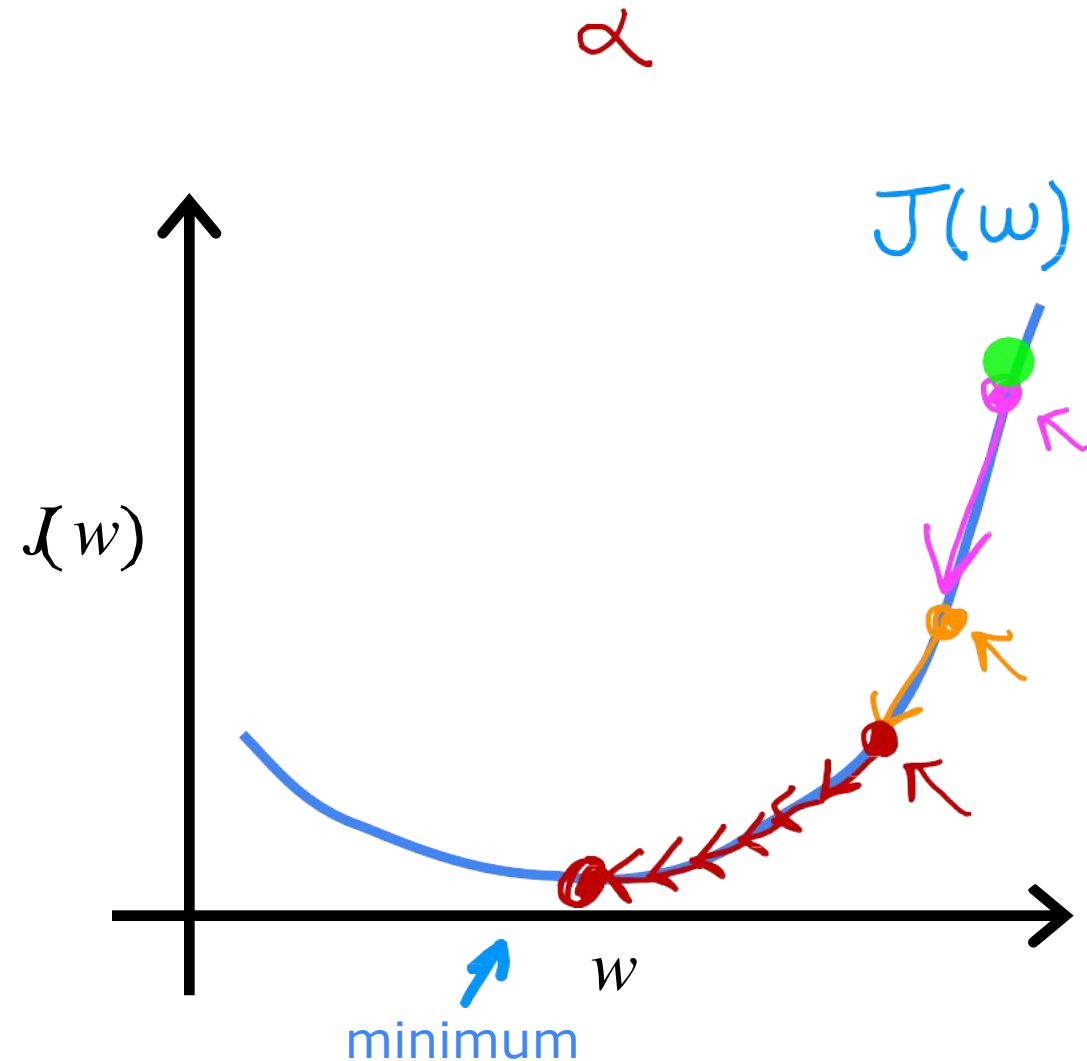
Can reach local minimum with fixed learning rate

$$w = w - \underbrace{\alpha}_{\text{smaller}} \underbrace{\frac{d}{dw} J(w)}_{\text{not as large}} \underbrace{J(w)}_{\text{large}}$$

Near a local minimum,

- Derivative becomes smaller
- Update steps become smaller

Can reach minimum without decreasing learning rate α





Gradient Descent for Linear Regression

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b) \rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

next slide
is optional!

$\frac{\partial}{\partial w} J(w, b) = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)})^2$
 (Optional)

$$= \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)}) \cancel{2} x^{(i)} = \boxed{\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}}$$

$\frac{\partial}{\partial b} J(w, b) = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)})^2$

$$= \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)}) \cancel{2} = \boxed{\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})}$$

no $x^{(i)}$

Gradient descent algorithm

$$\frac{\partial}{\partial w} J(w, b)$$

repeat until convergence

{

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

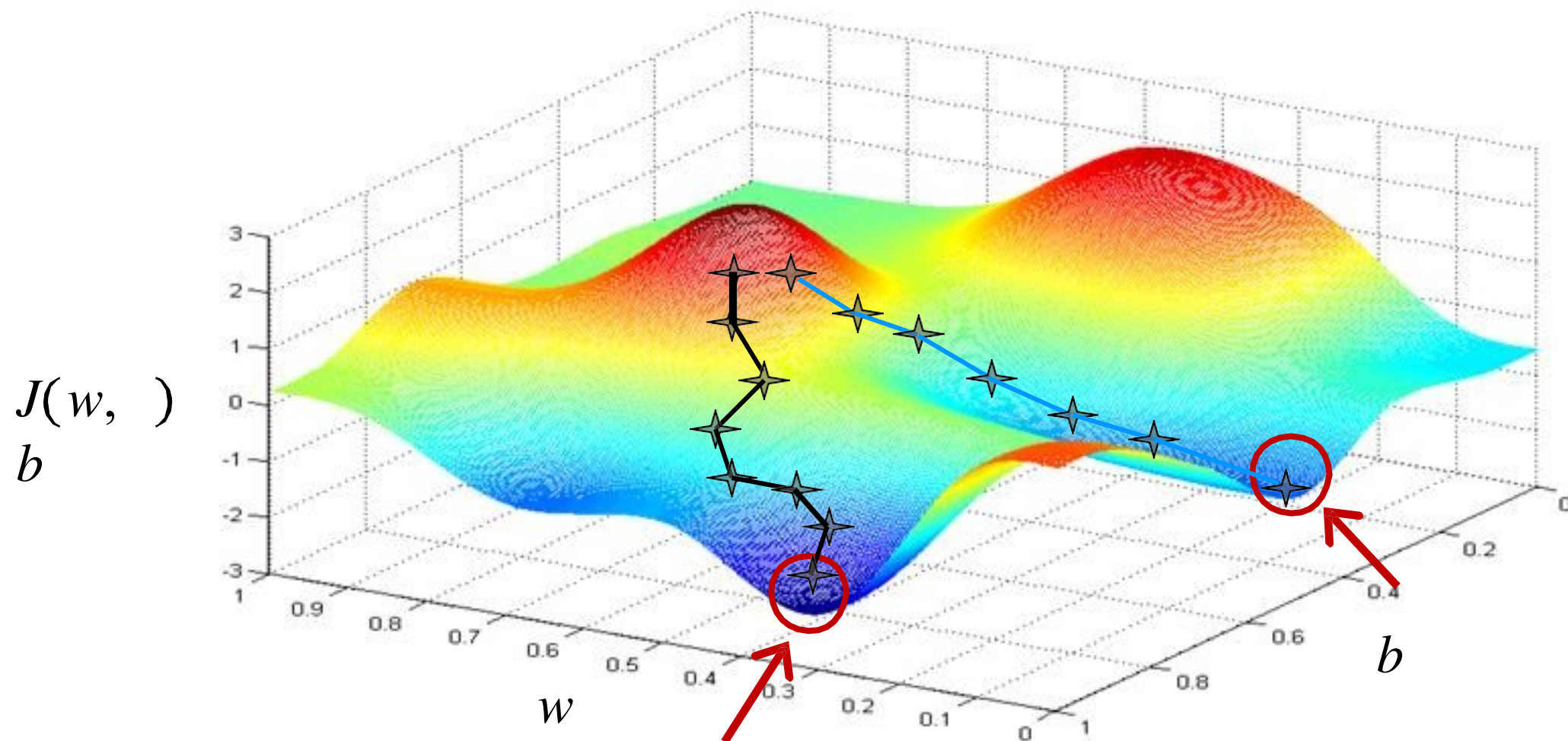
}

$$\frac{\partial}{\partial b} J(w, b)$$

Update
w and b
simultaneously

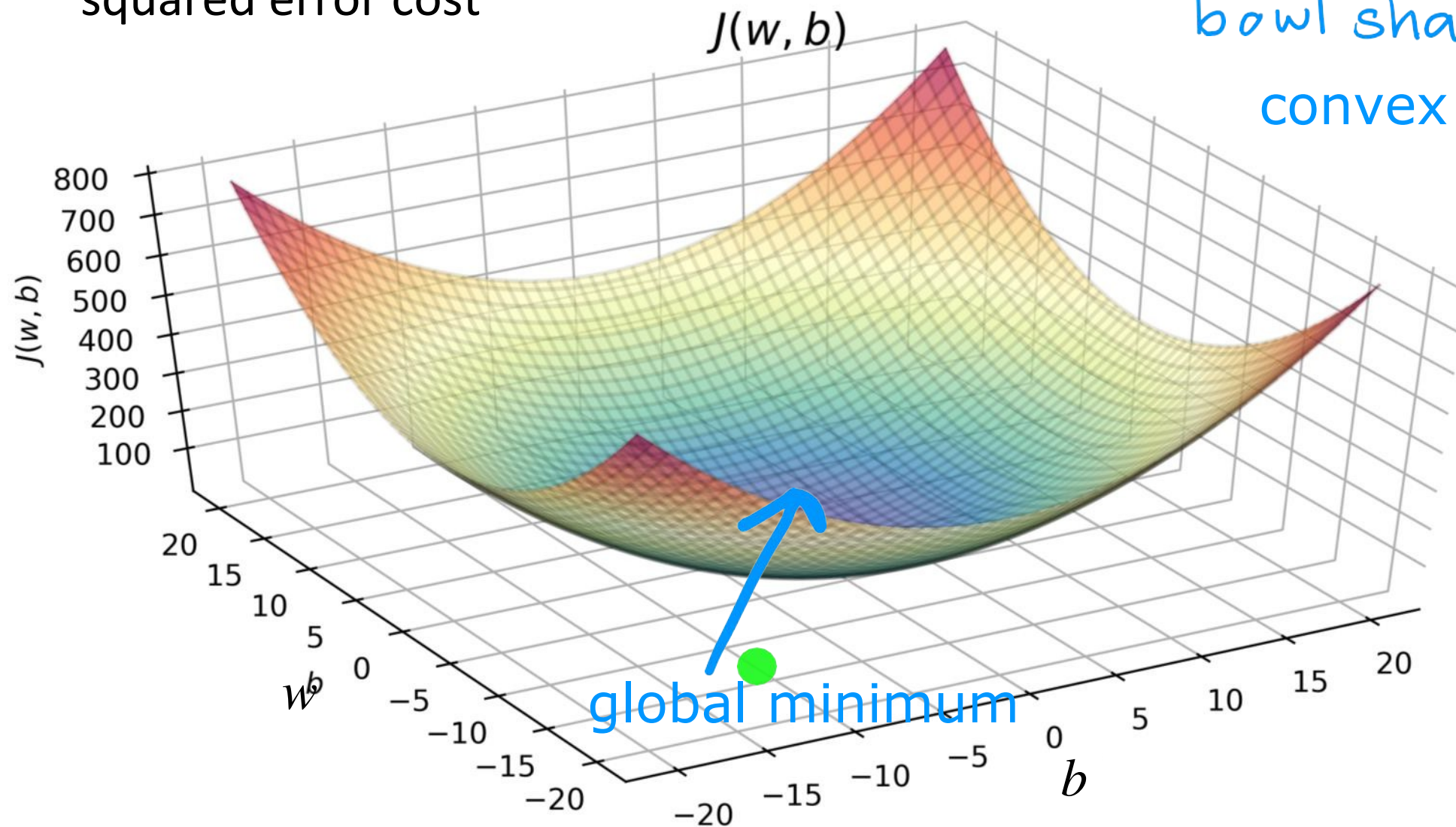
$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

More than one local minimum



squared error cost

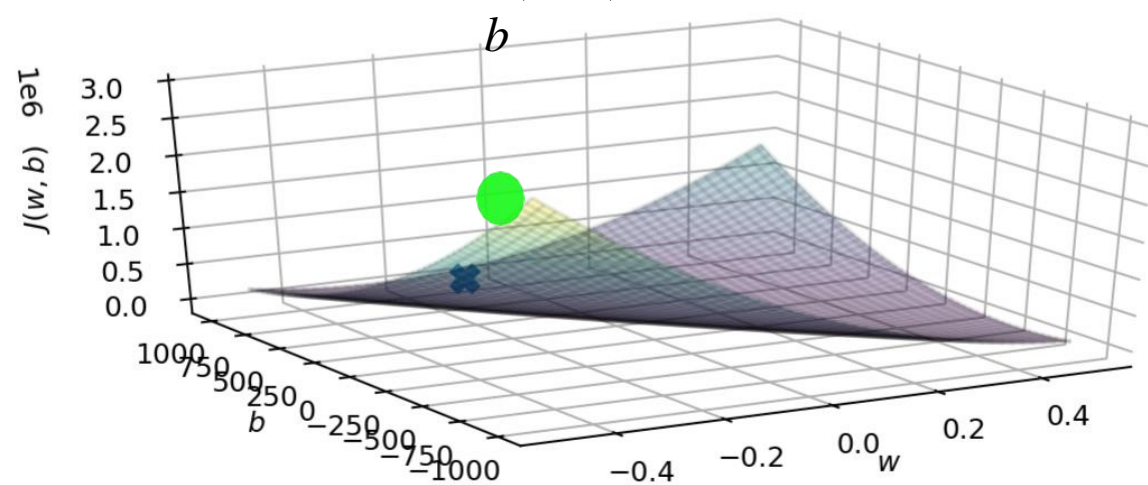
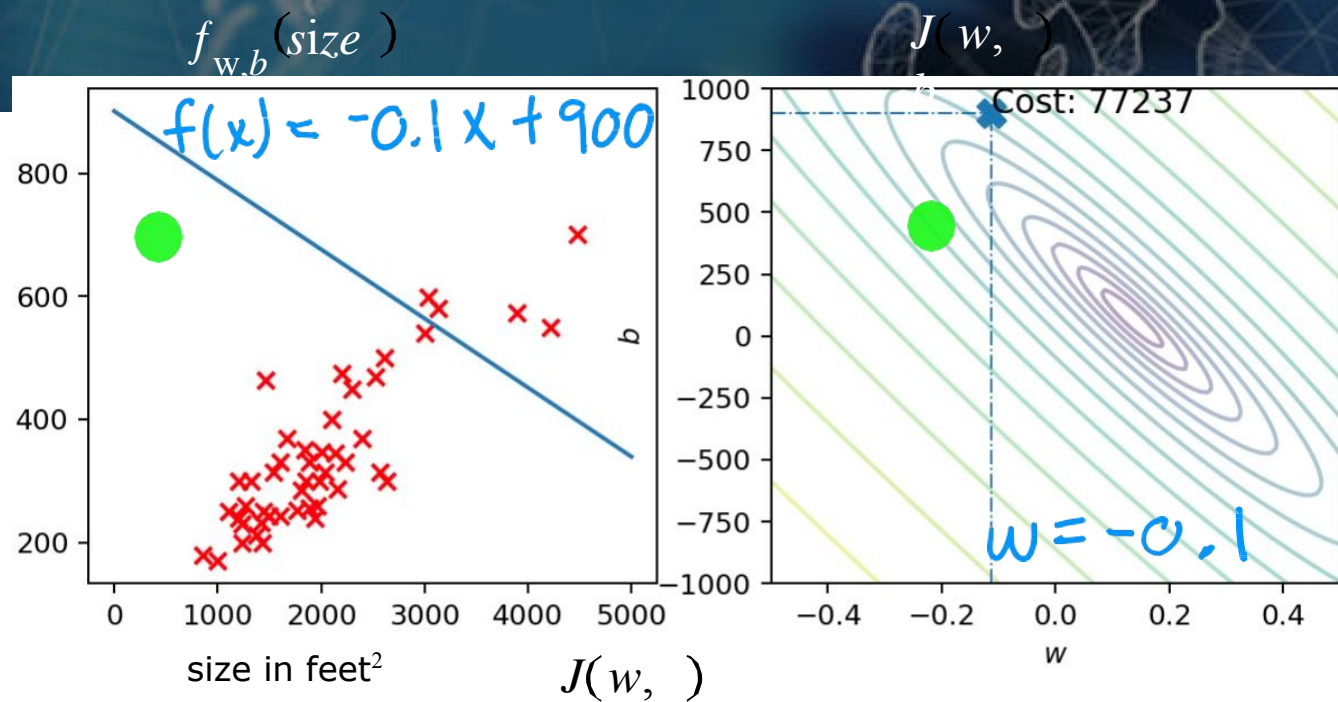
bowl shape 
convex function





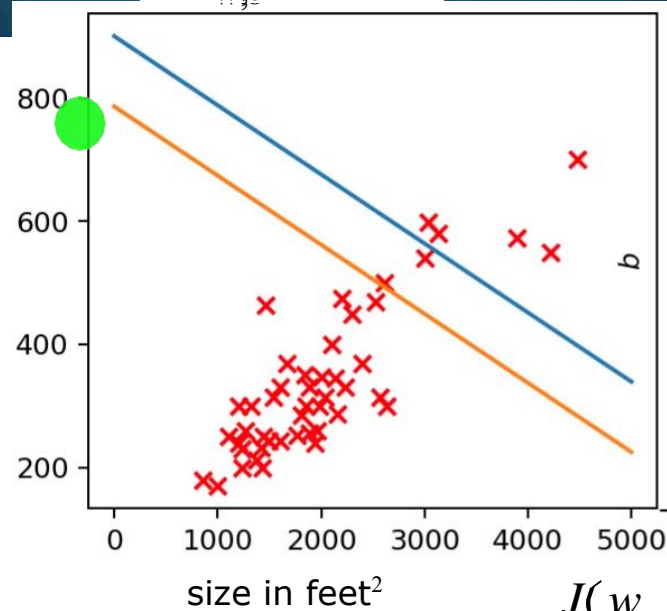
Running Gradient Descent

price
in
\$1000's

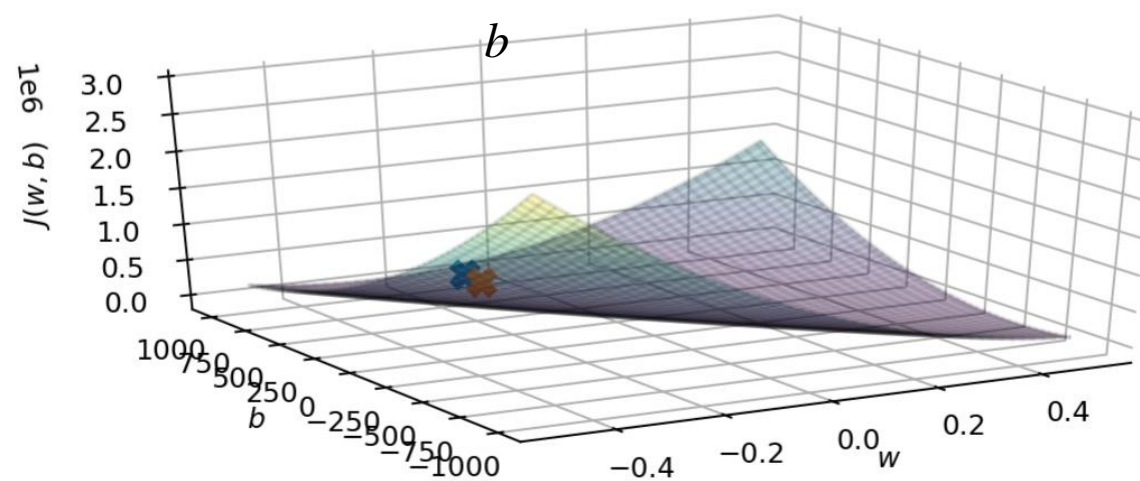
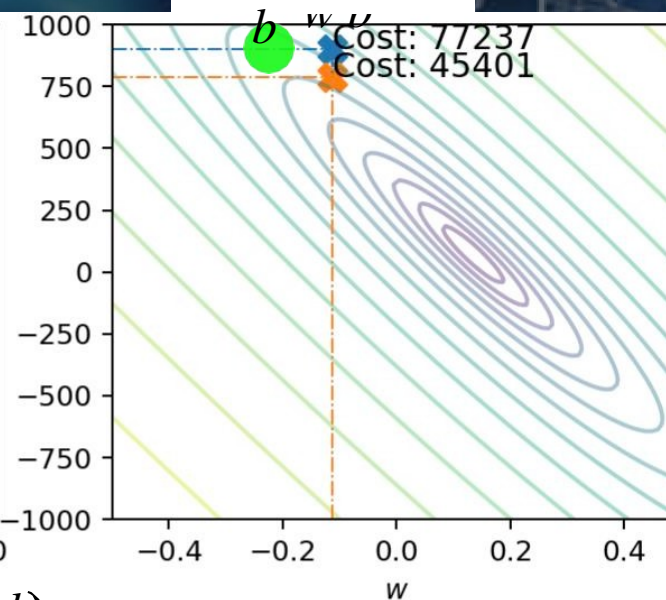


price
in
\$1000's

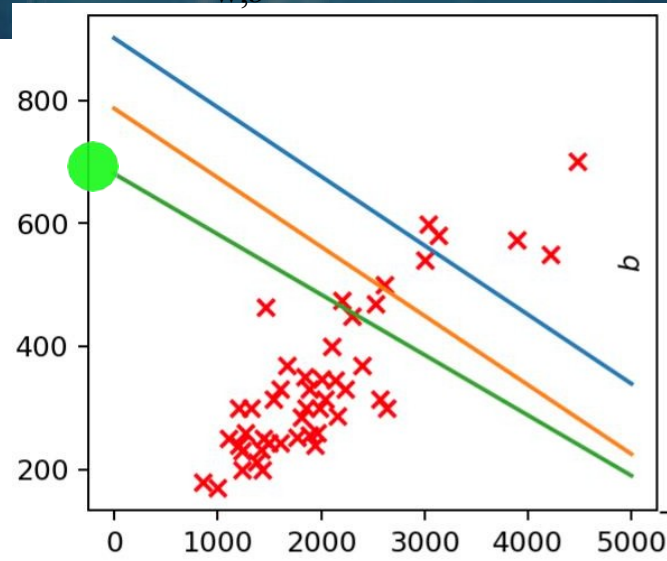
$$f_{w,b}(\text{size})$$



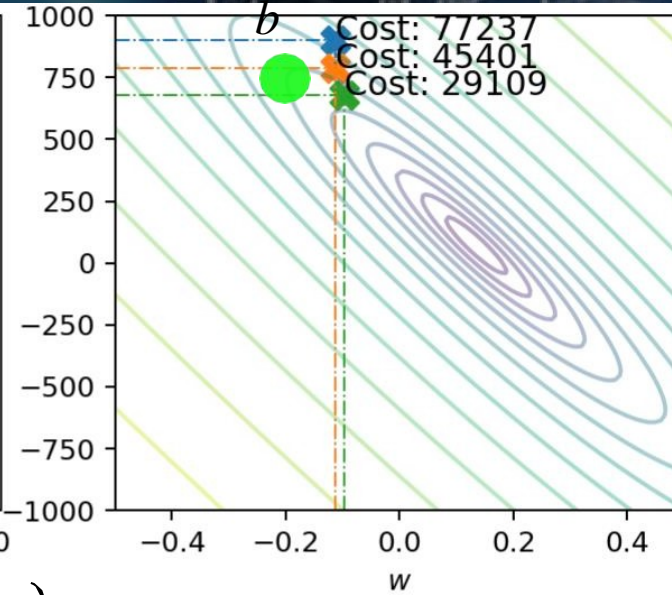
$$J(w, b)$$



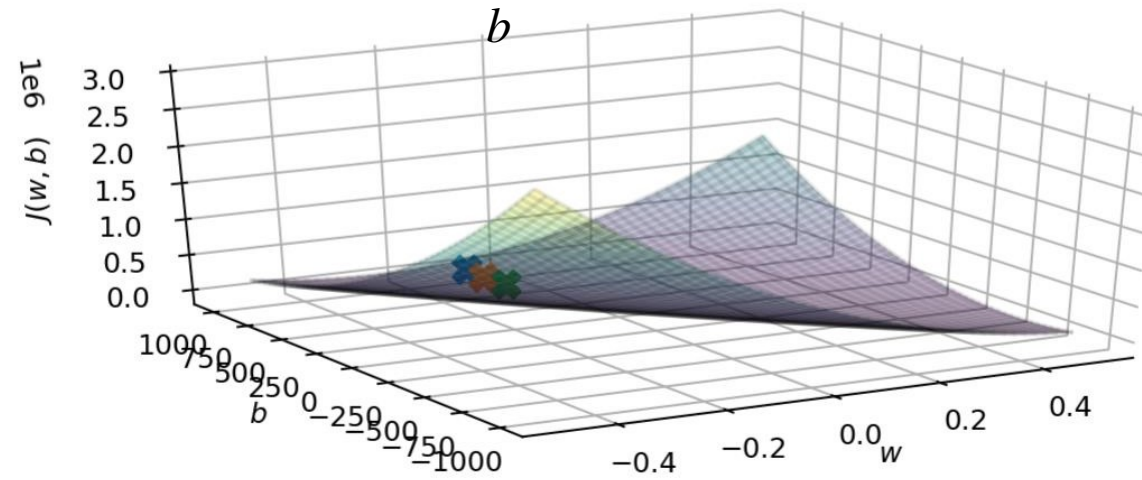
$f_{w,b}(\text{size})$



$J(w, b)$

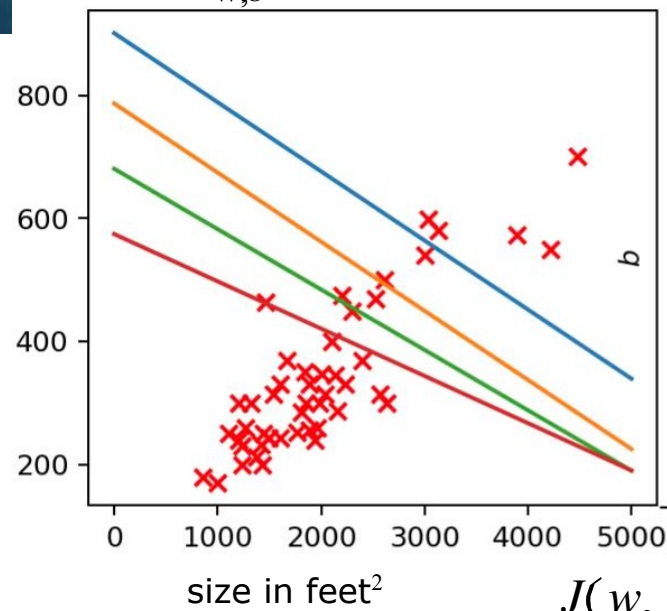


$J(w, b)$

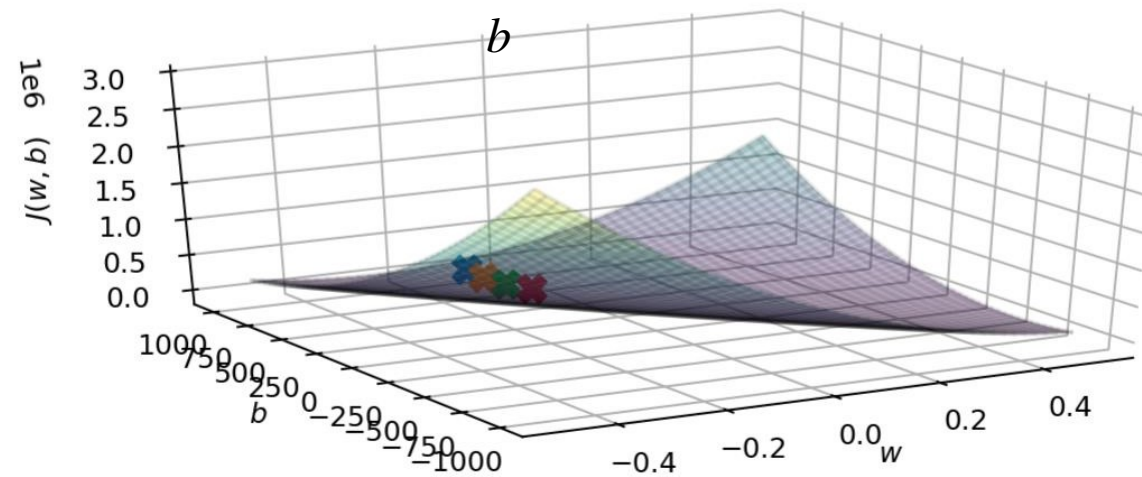
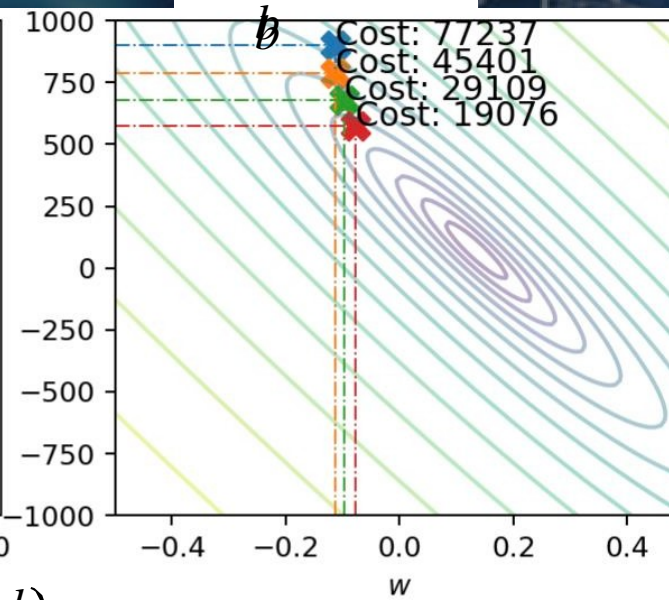


price
in
\$1000's

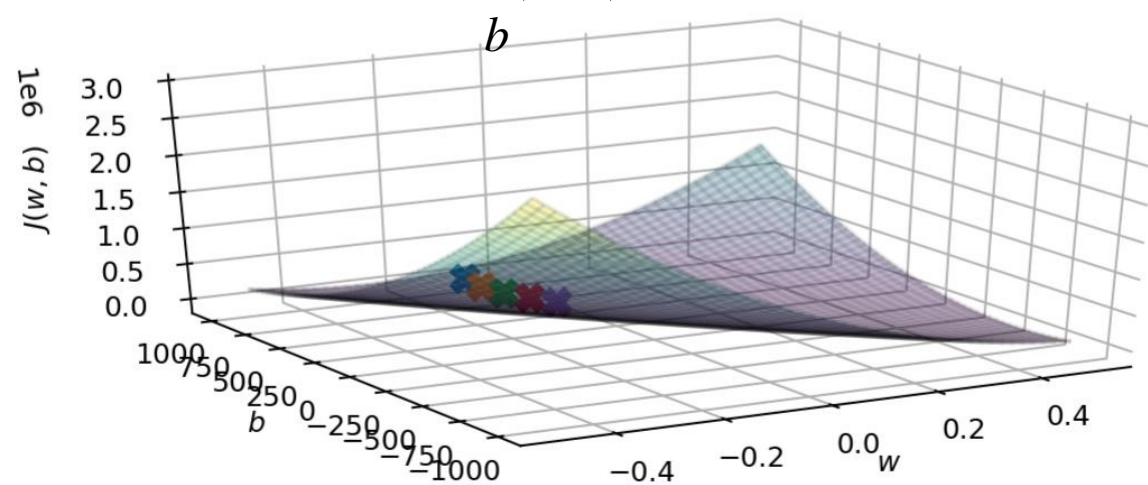
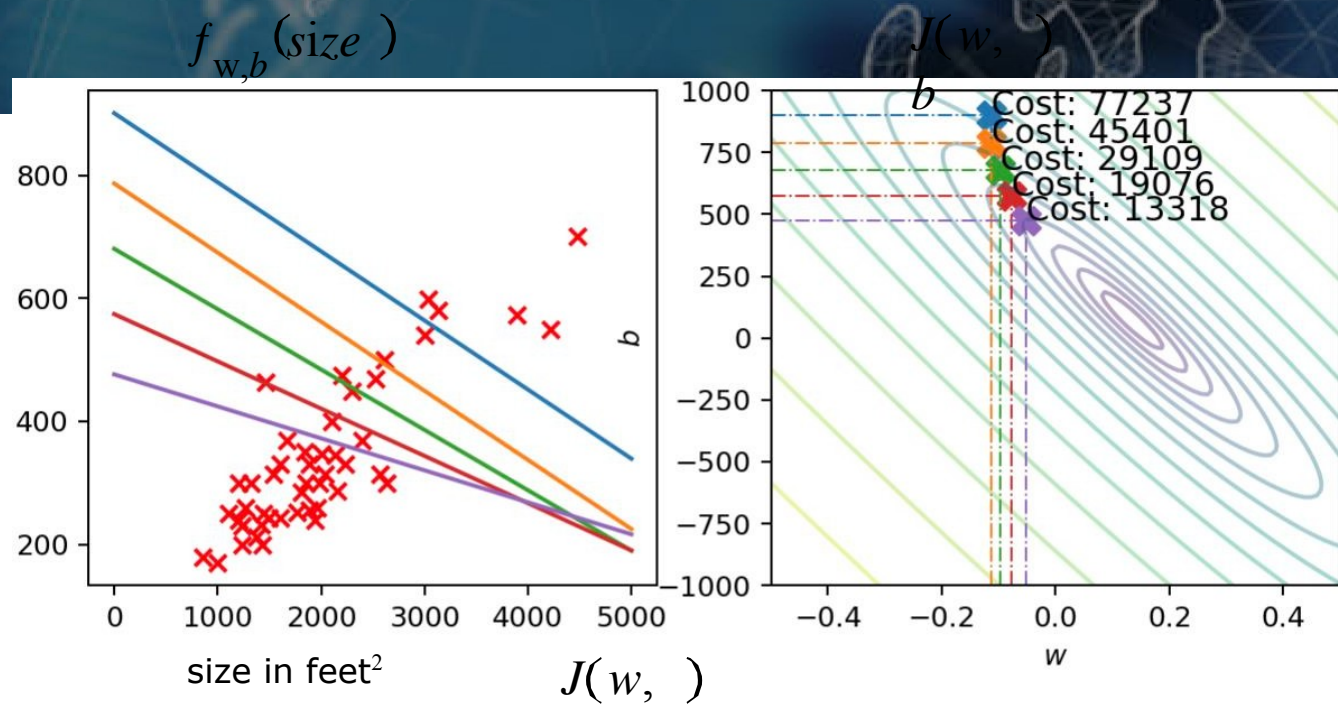
$$f_{w,b}(\text{size})$$



$$J(w, b)$$



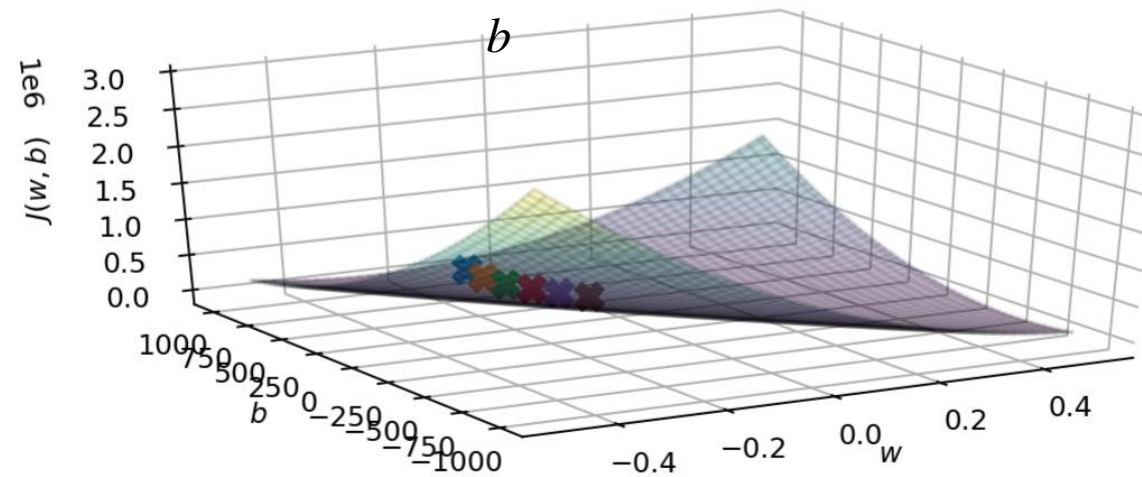
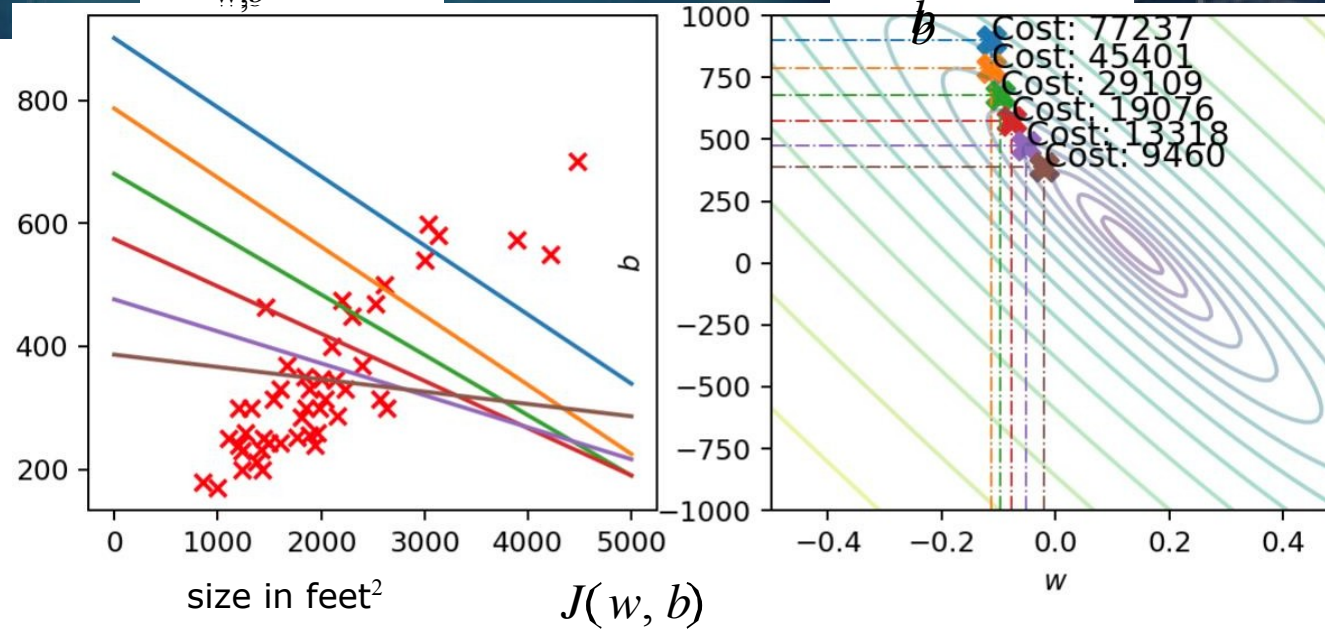
price
in
\$1000's



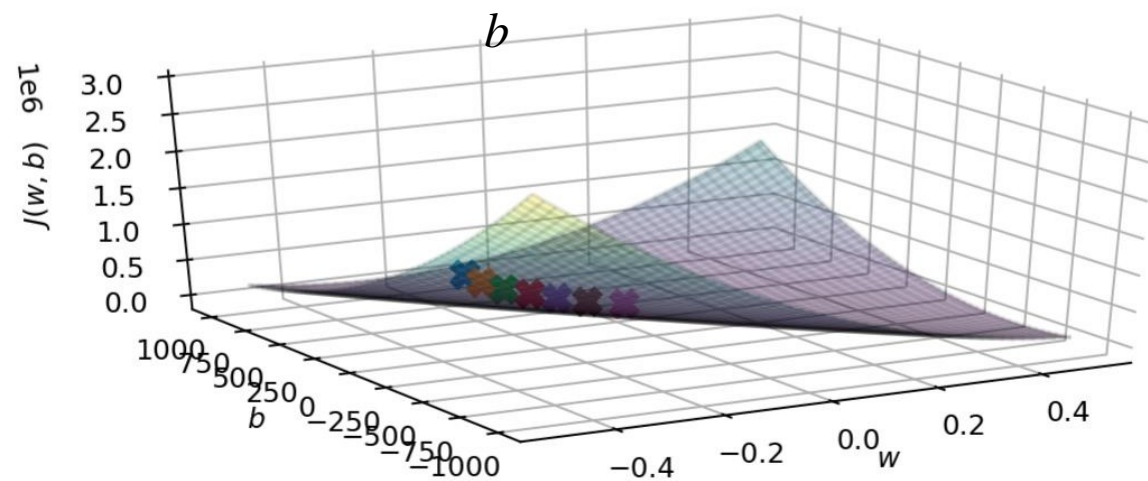
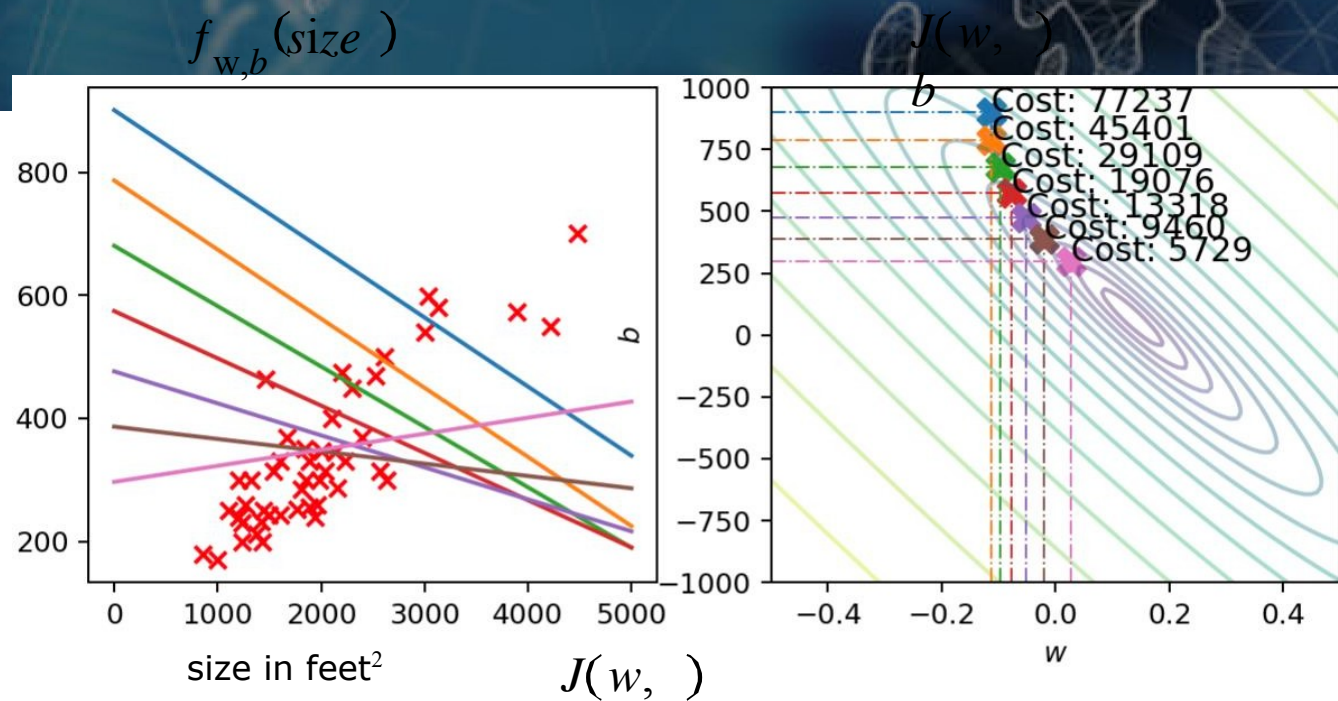
$$f_{w,b}(\text{size})$$

$$J(w, b)$$

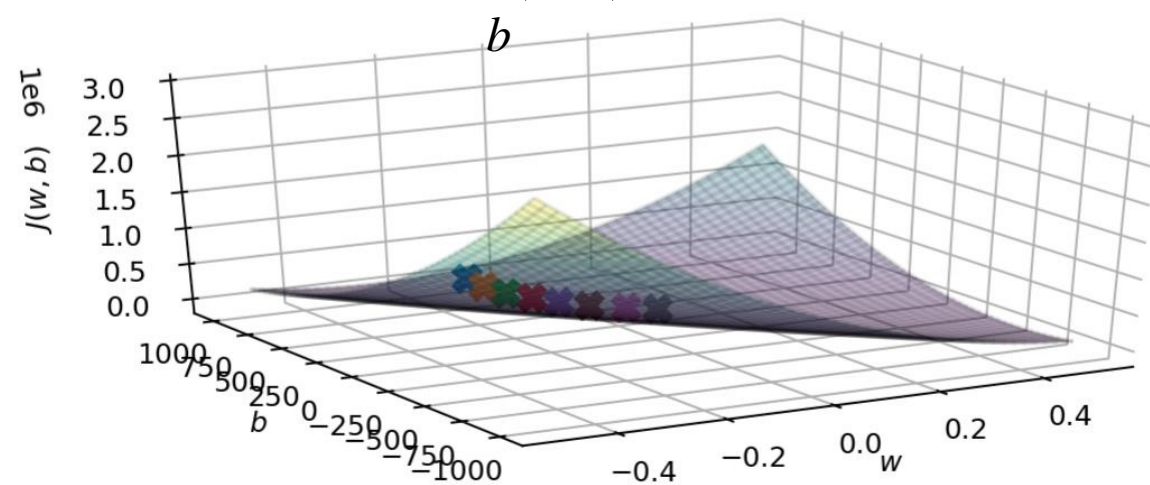
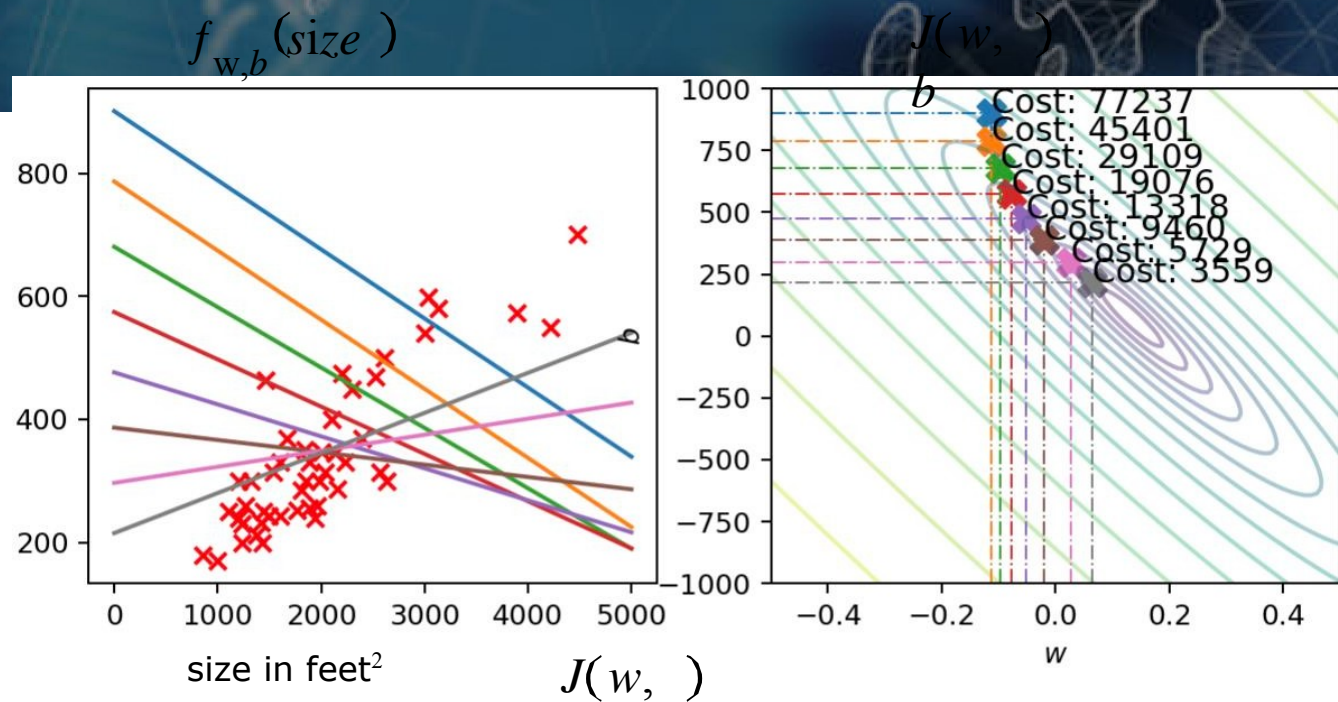
price
in
\$1000's



price
in
\$1000's

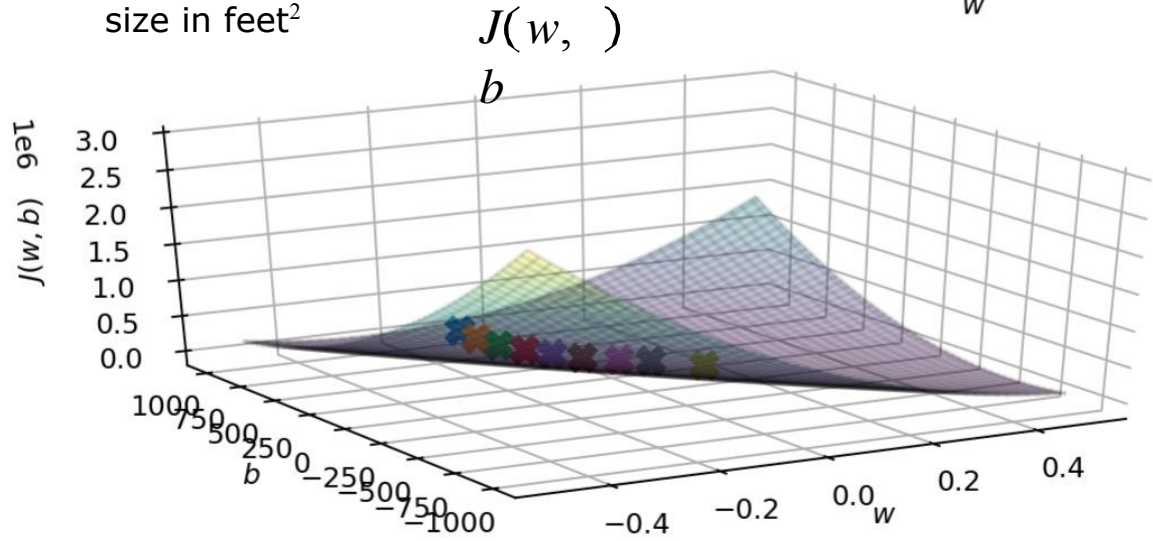
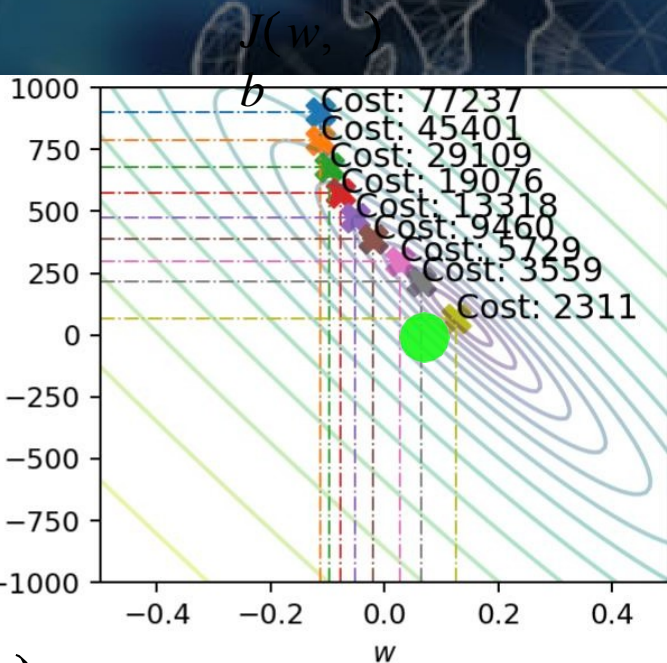
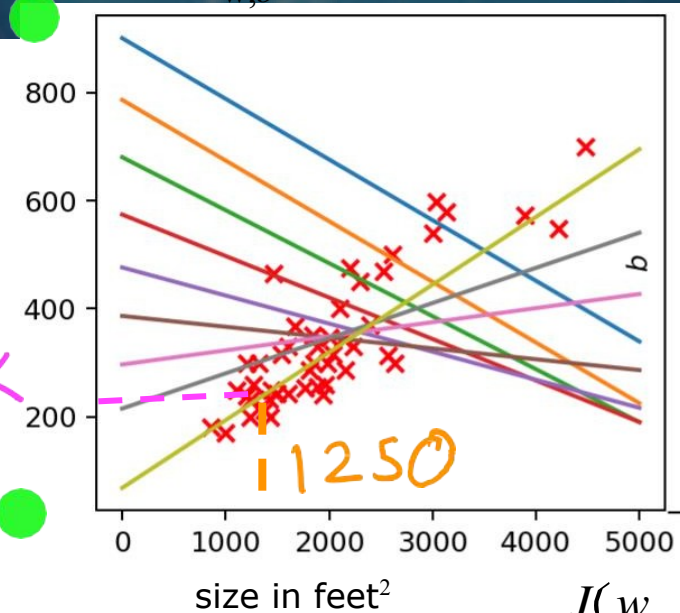


price
in
\$1000's



price
in
\$1000's
\$250K

$f_{w,b}$ (size)



“Batch” gradient descent

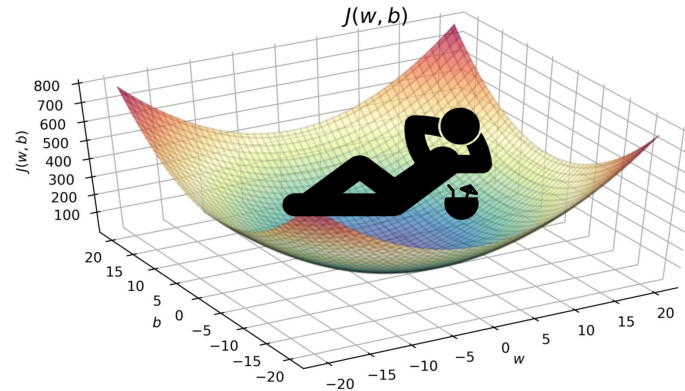
“Batch”: Each step of gradient descent uses all the training examples.

other gradient descent: subsets

	x size in feet ²	y price in \$1000's
(1)	2104	400
(2)	1416	232
(3)	1534	315
(4)	852	178
...	...	...
(47)	3210	870

$m = 47$

$$\sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$





Q & A